

Comprehensive Dockerization & Deployment Guide

FinCon & Financial Regulatory Hybrid RAG Architecture • Milestone 1 Production Manual

1. Executive Overview & Docker Production Strategy

Docker serves as the industry standard foundation for deploying production-grade AI systems. In complex RAG pipelines comprising PyTorch neural encoders, Cross-Encoder rerankers, ChromaDB vector stores, and LangChain orchestration, containerization guarantees **environment reproducibility**, **zero dependency drift**, and **frictionless portability** across local environments, Lightning AI Cloud, and enterprise Kubernetes clusters.

Key Architectural Advantage

Containerizing the FinCon RAG pipeline isolates CPU/GPU memory allocations, secures secrets via dynamic environment injection, and enables independent horizontal scaling of the FastAPI REST backend and the interactive Streamlit dashboard.

2. Pre-Dockerization Foundations (Codebase Readiness)

Before building Docker manifests, several prerequisite steps ensure the codebase is clean and build-ready:

- **Deterministic Dependencies:** Consolidate all core NLP, retrieval, and UI packages into a pinned requirements.txt.
- **Strict Secret Decoupling:** Ensure all API keys (Groq Cloud, OpenAI, Gemini) and database endpoints are loaded dynamically via os.getenv(...) using python-dotenv, preventing hardcoded credentials from entering images.
- **Persistent Storage Separation:** Separate immutable code from persistent state (raw datasets in ./data/raw, processed caches in ./data/processed, and HNSW vector graphs in ./chroma_db) for Docker volume mounting.

3. Writing Docker Manifests from Scratch

Step 3.1: Create .dockerignore

Prevents local virtual environments, checkpoints, and cached bytecode from inflating the image build context:

```
venv/  
.venv/  
__pycache__/  
*.pyc  
*.pyo  
*.log  
.ipynb_checkpoints/  
.env  
.git/
```

Step 3.2: Create Dockerfile

Defines the layered OS environment, C++ build dependencies, Python packages, and entrypoint:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1    PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends    build-essential curl git    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip &&    pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000 8501
CMD [uvicorn, api.app:app, --host, 0.0.0.0, --port, 8000]
```

Step 3.3: Multi-Service Orchestration with docker-compose.yml

Coordinates the FastAPI backend and Streamlit UI with persistent disk mapping:

```
version: '3.8'

services:
  api:
    build: .
    container_name: fincon_rag_api
    restart: unless-stopped
    ports:
      - 8000:8000
    env_file:
      - .env
    volumes:
      - ./data:/app/data
      - ./chroma_db:/app/chroma_db
    command: uvicorn api.app:app --host 0.0.0.0 --port 8000

  ui:
    build: .
    container_name: fincon_rag_ui
    restart: unless-stopped
    ports:
      - 8501:8501
    env_file:
      - .env
    volumes:
      - ./data:/app/data
      - ./chroma_db:/app/chroma_db
    command: streamlit run ui/streamlit_app.py --server.port 8501 --server.address 0.0.0.0
    depends_on:
      - api
```

4. Container Execution & Operational Lifecycle

Task / Stage	Terminal Command	Description
1. Data Ingestion	<code>docker compose run --rm api python -m src.ingestion.run_ingestion</code>	Parses 75k BAAI corpus, generates hierarchical parent-child chunks, builds ChromaDB & BM25.
2. Evaluation	<code>docker compose run --rm api python -m src.evaluation.evaluate_rag</code>	Runs benchmark test for Faithfulness, Correctness, Precision@K, Recall@K, and nDCG@K.
3. Service Launch	<code>docker compose up -d --build</code>	Builds production images and launches FastAPI (8000) and Streamlit (8501) in background.
4. Monitoring	<code>docker compose logs -f</code>	Streams real-time container logs across all microservices for query inspection.
5. Shutdown	<code>docker compose down</code>	Gracefully terminates all active containers while preserving persistent volumes.

5. Deploying & Using on Another Machine

To run the dockerized project on a remote cloud server, Lightning AI studio, or another workstation, choose one of two production strategies:

Method A: Git + Docker Compose Workflow (Recommended for Fast Testing)

1. On the new machine, clone the repository and enter the directory:

```
git clone
cd Milestone1_FinCon_RAG
```

2. Set up the local environment configuration:

```
cp .env.example .env
# Add your GROQ_API_KEY, etc. in .env
```

3. Build and launch all services with a single command:

```
docker compose up -d --build
```

4. Access the web services at `http://<machine-ip>:8501` (Streamlit UI) and `http://<machine-ip>:8000/docs` (FastAPI).

Method B: Container Registry (Docker Hub / ECR / GCR) Workflow

In enterprise deployments, images are pre-built once and distributed via a container registry without building from source on target servers:

```
# --- On Source Machine (Build & Push) ---
docker login
docker build -t your-username/fincon-rag:v1.0 .
docker push your-username/fincon-rag:v1.0

# --- On Target Machine (Pull & Run) ---
docker pull your-username/fincon-rag:v1.0
docker run -d --name fincon_api -p 8000:8000 --env-file .env -v /home/dell/Desktop/Echo INC./AI-Internship/Milestone1_FinCon_RAG/data:/app/data -v /home/dell/Desktop/Echo INC./AI-Internship/Milestone1_FinCon_RAG/chroma_db:/app/chroma_db your-username/fincon-rag:v1.0
```

Multi-Machine Portability Checklist

- Install Docker & Docker Compose on target host
- Mount persistent volumes for `./data` and `./chroma_db`
- Ensure firewall allows inbound traffic on ports 8000 and 8501.